EECS 376: Foundations of Computer Science **Due 8:00pm, March 27**
University of Michigan, Winter 2024 **Homework 8** **(accepted until 9:59 pm, no credit after)**

This homework has 8 questions, for a total of 100 points and 8 extra-credit points.
Unless otherwise stated, each question requires *clear*, *logically correct*, and *sufficient* justification to convince the reader.
For bonus/extra-credit questions, we will provide very limited guidance in office hours and on Piazza, and we do not guarantee anything about the difficulty of these questions.
We strongly encourage you to typeset your solutions in LaTeX.
If you collaborated with someone, you must state their name(s). You must *write your own solution* for all problems and *may not use any other student's write-up*.

(0 pts) 0. **Before you start; before you submit.**

- Read Handout 3: NP-Hardness Proofs to understand the components of such proofs.
- If applicable, state the name(s) and uniqname(s) of your collaborator(s).

Solution: Shikha Nukala, desigirl

(10 pts) 1. **Self assessment.**

Carefully read and understand the posted solutions to the previous homework. Identify one part for which your own solution has the most room for improvement (e.g., has unsound reasoning, doesn't show what was required, could be significantly clearer or better organized, etc.). Copy or screenshot this solution, then in a few sentences, explain what was deficient and how it could be fixed.

(Alternatively, if you think one of your solutions is significantly *better* than the posted one, copy it here and explain why you think it is better.)

If you didn't turn in the previous homework, then (1) state that you didn't turn it in, and (2) pick a problem that you think is particularly challenging from the previous homework, and explain the answer in your own words. You may reference the answer key, but your answer should be in your own words.

Solution: HW 7 Problem 3b
$= \{\langle G, k\rangle : G$ is an undirected graph with an independent set of size $k\}$.

An independent set in a graph is a subset C of the vertices for which the graph has no edge between any pair of vertices in C.
Solution:
First we need to verify that has a verifier for c = some k. We can do this by constructing a verifier V for .
V = On input $\langle G, k\rangle$:
1) non deterministically select k vertices in G
2) check if those k vertices form an independent set in G to create subset C
3) if there is such a solution, accept, otherwise reject
$\langle G, k\rangle$ in , if there is a set with k vertices, D accepts
$\langle G, k\rangle$ not in , if there is not a set with k vertices, D rejects
From this we can see that the algorithm is verified, accepting or rejecting on any possible

input k and graph G. Next we have to prove there is a nondeterministic polynomial time algorithm that runs with respect to the size of input k. From this algorithm, we can see that the algorithm runs with respect to the size of the graph and choosing k vertices, then iterating through those k vertices ensuring that the resulting subset is an independent set or not, therefore the complexity can be worse case represented as $O(n^2)$ therefore this is a nondeterministic polynomial algorithm.

This solution is partially incorrect because the algorithm provided in the solution is to brute force the solution, not to use the verifier on an existing solution. Here the algorithm in the verifier does not take in certificate c, and does not use the certificate to check if it is a viable solution given the conditions of the problem. Instead of selecting k vertices in graph G, we should use a certificate c to check if the cardinality of c is k. That would indicate that the solution indeed has k vertices. Next would be to check if c is indeed a subgraph in G. Once those two conditions are met, then check the last condition of the independent set to see if the pair of vertices are in G. If they are then reject c because it is not an independent set, otherwise accept c. This problem was read incorrectly and implemented not as a verifier problem but as a decider problem.

2. **Review of last week's materials.**

(4 pts) (a) Define the terms P, NP, NP-Hard, and NP-Complete. (You do not need to define the terms that are used in these definitions.)

Solution: P: stands for polynomial time. It is a complexity class with a set of decision problems (languages) that can be solved by some deterministic (if there is only one possible action each step) machine in polynomial time. Decided in polynomial time.
NP: stands for non-deterministic polynomial time. It is a complexity class it a set of decision problems (languages) that can be solved by some non-deterministic (if there is more than one by finite number of actions that can be taken each step) machine in polynomial time. Verified in polynomial time.
NP-Hard: a problem that is at least as hard as every problem in NP.
NP-Complete: a problem that is the hardest in NP. That is, a problem that is still in the complexity class NP but is at least as hard as every problem in NP.

(8 pts) (b) For each of the following statements, state, with brief justification, that it is *known to be true*, *known to be false*, or its truth/falsehood is *unknown*. Note: as always, if a statement has any counterexample(s), it is false.

(i) Let V be an efficient verifier for $L \in \mathsf{NP}$ and x, c be strings; if $V(x, c)$ accepts, then $x \in L$.

(ii) Let V be an efficient verifier for $L \in \mathsf{NP}$ and x, c be strings; if $V(x, c)$ rejects, then $x \notin L$.

(iii) If $\mathsf{P} \neq \mathsf{NP}$ and L is NP-Complete, then $L \notin \mathsf{P}$.

(iv) If L is NP-Hard, then $L \notin \mathsf{P}$.

Solution: i) This is a problem that is known to be true. If V(x,c) accepts, then we know that there must be a problem that runs in polynomial time as the verifier for $L \in \mathsf{NP}$ means that it is verifying that all languages L run in polynomial time. Therefore if there exists a certificate c that causes V to accept x, then x must run in polynomial time and be in NP.
ii) Truth is unknown. If V(x,c) rejects x, this could be due to V not recognizing x as a language in L or could be because a particular certificate c does not verify x as a valid language to V. V rejecting x can occur as long as V is not a correct/complete verifier of $L \in \mathsf{NP}$, so it can occur. Than x is indeed in L even if rejected by V.
iii) Known to be true. We know that L is at least as hard as every problem in NP. Therefore if L were to be in P we know that all languages in L must also be in NP, so this would not work if $\mathsf{P} \neq \mathsf{NP}$ as P would have to equal NP for this condition to hold. Thus L cannot be in P.
iv) Truth is unknown. While it seems correct that if L is NP-Hard then some of the languages or all are not likely to be solved in polynomial time as they are either the most hard in NP or are not in NP. But this cannot be proved as we do not know if whether P = NP or not and therefore cannot say whether this statement is always true or not even if it is mostly true.

(4 pts) (c) Briefly explain what the "Boolean satisfiability" (SAT) problem is, and give one example instance that is satisfiable, and one that is unsatisfiable. Each example must include at least 2 variables and at least 3 (not necessarily distinct) logical operators (AND/OR/NOT).

Solution: The Boolean satisfiability problem determines whether a Boolean formula is satisfiable or unsatisfiable, meaning whether there exists truth values for each variable in the formula that make the overall formula true or not. An example of a formula that can be satisfied is $((x \wedge y) \vee (x \wedge z))$. From this formula we can see that for the case of x,y,and z are all the same truth value (ex. if x, y, and z all true), then the total statement is true. For unsatisfiable, we can use the example of $(x \wedge y) \wedge (\neg x \wedge y)$. From this example we can see that if x and y amounts to true, then not x and y cannot amount to a true value as y cannot by both a true and false value. This means that neither of these statements together can ever be true, thus the formula does not have a satisfiable solution.

(4 pts) (d) Towards proving the Cook-Levin Theorem in lecture, we outlined a proof of the following statement:

> Let language $L \in \mathsf{NP}$ by arbitrary, and fix an efficient verifier VerifyL for L. Given any instance x of L, in polynomial time we can construct an instance ϕ of SAT such that ϕ is satisfiable *if and only if* there *exists* a c such that the tableau of VerifyL(x, c) has q_{accept} in it.

Briefly explain why proving this statement proves that SAT is NP-Hard.

Solution: To prove the statement from lecture, we found that the boolean formula was satisfiable if and only if there existed a certificate c such that V_L(x,c) accepts the formula with an instance x of L and the certificate c is encoded by the boolean formula. Therefore we knew that if there is a certificate c such that the verifier accepts, then the boolean formula is satisfiable and if there is not, the formula is unsatisfiable. In NP, we know any lannguage can be reduced to SAT by applying a boolean formula to the language. Therefore, from the proof above we know that any language in NP can be reduced to SAT in polynomial time to be verified. Therefore SAT is NP-Hard as each boolean problem can be as hard or harder than the connecting NP problem.

(10 pts) 3. **Turing reductions vs. poly-time mapping reductions.**

Suppose that $\mathsf{P} \neq \mathsf{NP}$, and let $A \in \mathsf{P}$ be arbitrary. Prove that there is a *Turing* reduction from SAT to A (i.e., SAT $\leq_T A$), but there is *no polynomial-time mapping reduction* from SAT to A. (i.e., SAT $\not\leq_p A$).

Solution: Since we know that $A \in \mathsf{P}$, we can say that there exists a polynomial time machine M than decides A. Therefore to construct a Turing reduction from SAT to A we first simulate the polynomial time verifier for SAT using M with an instance of SAT b.
V(b, c) with input b
for each certificate c (which is a truth assignment for b), have b incorporate c and run M to decide whether c is valid for b.
if M accepts, then b is satisfiable
if M rejects, b is unsatisfiable
We know that M is polynomial time and we can see from the verifier above that the verifier for SAT is polynomial time, so we have that the reduction from SAT to A is a turing reduction.
To disprove the second statement, if we assume that there is a polynomial mapping reduction from SAT to A¡ then we would have a polynomial time algorithm that maps instances of SAT to instances of A. However, since SAT is NP-Hard, there cannot be a polynomial time algorithm that solves SAT unless P = NP, which we do not hold to be true. Thus we cannot solve for SAT by reducing it to polynomial time A. Therefore this mapping cannot have a relation.

4. **Subset sum.**

Consider the following reduction involving 3SAT and the language

$$\textsc{SubsetSum} = \{(A, s) : A \text{ is a } \textit{multiset} \text{ of integers} \geq 0, \text{ and } \exists\, I \subseteq A \text{ such that } \sum_{a \in I} a = s\}.$$

A *multiset* A is just like a set, but it can have duplicate elements. A submultiset $I \subseteq A$ also can have duplicate elements, as long as it does not have more copies of an particular element than A does.[1]

The reduction is given a 3CNF formula φ with n variables $x_1, \ldots, x_n$ and m clauses $c_1, \ldots, c_m$. We assume without loss of generality that there is no repeated clause. (Recall that each clause is the OR of exactly three literals, where a literal is either some variable x_i or its negation $\overline{x_i}$.)

The reduction constructs a collection of numbers as follows:

1. For each clause c_j, it constructs integers a_j and b_j, each $n+m$ decimal digits long, where the jth digit of both a_j and b_j is 1, and all other digits are 0.
2. For each variable x_i, it constructs integers t_i and f_i, each $n+m$ decimal digits long, where:
   (a) The $(m+i)$th digits of both t_i and f_i are 1.
   (b) The jth digit of t_i is 1 if literal x_i appears in clause c_j.
   (c) The jth digit of f_i is 1 if literal $\overline{x_i}$ appears in clause c_j.
   (d) All other digits of t_i and f_i are 0.
3. It constructs a target sum s that has $n+m$ decimal digits, where the first m digits are 3 and the last n digits are 1.

The reduction outputs (A, s), where A is the multiset of all the numbers a_j, b_j, t_i, and f_i.

For example, for the formula $\varphi = (x_1 \vee \overline{x_2} \vee x_3) \wedge (\overline{x_1} \vee x_2 \vee \overline{x_3})$, the reduction constructs:

Number	$j=1$	$j=2$	$i=1$	$i=2$	$i=3$
a_1	1	0	0	0	0
b_1	1	0	0	0	0
a_2	0	1	0	0	0
b_2	0	1	0	0	0
t_1	1	0	1	0	0
f_1	0	1	1	0	0
t_2	0	1	0	1	0
f_2	1	0	0	1	0
t_3	1	0	0	0	1
f_3	0	1	0	0	1
s	3	3	1	1	1

[1] In class we defined a slightly different version of SubsetSum with ordinary sets instead of multisets. With a little more work, the version with ordinary sets can also be proved NP-Complete.

Answer the following:

(2 pts) (a) Fill in the blanks: the above reduction is for showing that _____ $\leq_p$ _____.

> **Solution:** The above reduction is for showing that 3SAT $\leq_p$ SubsetSum.

(15 pts) (b) Prove that the reduction is correct by proving the following, for any 3CNF formula φ:

- $\varphi \in$ 3SAT $\implies (A, s) \in$ SubsetSum;
- $(A, s) \in$ SubsetSum $\implies \varphi \in$ 3SAT.

> **Solution:** To prove that the reduction is correct, we can start with $\varphi \in$ 3SAT $\implies (A, s) \in$ SubsetSum. If we assume that φ is satisfiable in 3SAT, then there exists a truth assignment that satisfies φ. For each clause c_j we know that there are integers $a_j and b_j$ where the jth digit is 1 and all other digits are 0 by the reduction. Then we have integers $t_i and f_i$ such that the (m+i)th digit is 1 if the condition for each variable x_i appears in c_j. Knowing these, we know that c_j contributes to the sum because there is a jth digit of 1 for integers in c_j and that each variable contributes to the sum through the 1s in $t_i and f_i$. The target sum has first m digits being 3 and last n digits as 1, so A is constructed of all these numbers, and from the example above we know that it is possible to select a subset out of A such that the sum of elements in the set equals s, and we know that there can be a sum of 3 in every m digits because each digit impacts the sum at least once where applicable.
> To prove $(A, s) \in$ SubsetSum $\implies \varphi \in$ 3SAT, if (A,s) is in SubsetSum, then there should exist some subset in A such that the sum of elements in the subset equals s. We know that each element in A is either a clause c_j or a variable x_i and whichever variables are true and whichever clauses are satisfied will be selected from A. We can construct numbers in A like in the example such that some elements combination of clauses and variables that are satisfied and true will sun to s. Therefore there should exist a truth assignment for the clauses in φ that makes φ satisfiable to achieve the solution as given in the example. Therefore both statements are proved with the given turing reduction.

(5 pts) (c) Why do we set the first m digits of s to be 3? Would the reduction still be correct if we used 2 instead? What about 4?

> **Solution:** To set the first m digits to 3 is to ensure that the target sum s is is distinct from the sums of the integers that come from the clauses and variables. If one of the sums of elements in the subset that is of the solution matches one of the integer sums, then there could be a false positive in the solution. For the number 2, this is a sum that can be achieved by the clauses and variables. If the number 4 is used, the first m digits have to be constructed by numbers that have at least one digit as 1, therefore there should be no overlap in the first m digits if s is set to 4.

(18 pts) 5. **Tours and cycles.**

Recall the following definitions (in this problem, all graphs are undirected):

$$\text{HamCycle} = \{G : G \text{ is an unweighted graph with a Hamiltonian cycle}\}$$
$$\text{TSP} = \{(G, k) : G \text{ is a weighted, complete graph with a tour of weight} \leq k\} .$$

Prove that HamCycle $\leq_p$ TSP. (Recall from lecture and the handout what such a proof involves.) Since HamCycle is NP-Hard, is follows that TSP is also NP-Hard.

Reminder: an instance of HamCycle is an arbitrary unweighted graph, while an instance of TSP is a *complete* graph where each edge has a weight.

Solution: To prove that HamCycle $\leq_p$ TSP, there needs to exist a polynomial time reduction. If define the reduction to be f(G(V,E), G_1(V,E)), then we have:
f(G(V,E)):
1) for each edge e between two any vertices in the graph G, there can be a weight of 1 to the same corresponding edge e_1 in a new graph G_1. Therefore all edges in G_1 should be of weight 1.
2) For each vertex v in G, add a vertex v_1 to the graph G_1. This new vertex should connect to every vertex remaining in G_1, and the edges of these new connects will have weight 2.
3) return G_1
G_1 is made in polynomial time because there is one iteration over each vertex and for each vertex there is an edge added for every other vertex. Therefore this is exponential time or $O(n^2)$ for n vertices.
To support the reduction, if G has a HamCycle then from the reduction we can see that G_1 has weight of $\leq 2|V|$. Then for G we can go through G such that we visit every vertex exactly once and the last vertex is adjacent to the first as per a cycle. That means that in the graph G_1 which holds the G cycle, we can go through all vertices v_1 as we did in G and return back to the starting vertex on an edge of weight 2. Therefore the weight of the full traversal would be $\leq 2|V|$. Additionally, the other way around we can see that if we go from complete graph G_1 which weight $\leq 2|V|$, the G is a HamCycle because from the reduction we know that G_1 has edges of weight 1 but connects vertices with edges of weight 2, and therefore removing the extra vertices of weight 2 edges would restore the cycle G with all edges of weight 1 which should contribute to an unweighted graph.Therefore we can see that since HamCycle is NP-Hard and there exists turing reduction HamCycle $\leq_p$ TSP, TSP is also NP-Hard.

6. **More Knapsack!**

Recall the 0-1 knapsack problem: an instance is a list of n item weights $W = (W_1, W_2, \ldots, W_n)$, their corresponding values $V = (V_1, V_2, \ldots, V_n)$, and a weight capacity C. (All values are non-negative integers.) The goal is to select items having maximum total value, such that their total weight does not exceed the capacity. We can make this a decision problem by introducing a "budget" K and asking whether a total value of at least K can be achieved (again, subject to the capacity constraint):

$$\text{Knapsack} = \{(W, V, C, K) : \exists\, S \subseteq \{1, \ldots, n\} \text{ such that } \sum_{i \in S} W_i \leq C \text{ and } \sum_{i \in S} V_i \geq K\}.$$

(14 pts) (a) Prove that KNAPSACK is NP-Hard, by showing that SUBSETSUM $\leq_p$ KNAPSACK. (See Question 4 for the definition of SUBSETSUM.)

Solution: First we must show that there is a turing reduction SUBSETSUM $\leq_p$ KNAPSACK.
We can start by defining the reduction as f(A,s) as the input subset sum,
f(A,s):
1) for knapsack problems we need to have the list of weight W, their values V, a weight capacity C, and a total value of at least K. There for we define W to be the list of weight of each item in A.
2) Then the list V can be defined as the item values where each value is the same as the weight because we are taking from A, thus we have W = V.
3) Then we define C = s as the capacity of the weight should be the target sum of the subset sum.
4) Since the weights and values derived from A are the same, we also set K = s because it is the least total value the items can add up to.
5) return knapsack (W,V,C,K)
We can see from the previous subset problem in problem 4 that the problem runs in polynomial time. Additionally, we can see from this reduction that it linearly goes through all values in A and adds them to W and V respectively, therefore we can consider this algorithm as polynomial time as well.
To prove the reduction, first there much exist a subset in A such that the items weight adds up to s as defined in the knapsack problem. Therefore since we know that W and V should map to the same values, we need to show that some combination of weights in a add to s. We can do this by selecting the items for the knapsack using the knapsack algorithm, and ensure the weight does not exceed C but is equal to C as it also has to be at least K. Since this means the sum can only be equal to C and K and therefore equal to s. Thus in the case that there is a subset in A that equals s, there is also a knapsack that is a valid instance from A.
If there is not a valid subset in A such that the sum of values equals to s, then there is no instance of W or V to equal C or K, which means there is no valid knapsack (W,V,C,K) that is from this subset sum instance. Since this reduction works both ways, it is correct and knapsack is NP-Hard as is subset sum.

(6 pts) (b) Recall that we previously gave a dynamic programming algorithm that solves KNAPSACK in $O(nC)$ time. Does this prove that P = NP? Why or why not?

Solution: No it does not because the knapsack problem in $O(nC)$ time is optimized to solve the problem and is one instance of a solution for the knapsack problem. While this instance may be in polynomial time as a decision problem through dynamic programming, the instance this problem refers to is an optimized problem, not just a decision problem, which is the criteria for the problems in NP. Not all problems in NP can be solved efficiently as the knapsack problem has a optimization. P = NP would only be true if every decision problem in NP was solved in polynomial time which is not the case.

(8 EC pts) 7. **Optional extra-credit question: Network reliability.**

The Republic of Alvonia owns the internal internet infrastructure for the country and leases out connections to Internet Service Providers (ISPs). We represent that network as an undirected graph. Assume that each edge in the graph has a positive integer *rental cost* associated with it. An ISP is confronted with the problem of spending the minimum amount of money on link rental so that it can provide adequately reliable service to its customers.

Here is how we quantify reliability: We say that two paths in the network are *disjoint* if they have no vertices in common, except for possibly their endpoints. For example, there can be two disjoint paths from vertex v_{42} to v_{100}, but v_{42} and v_{100} can be the only vertices that these paths have in common (since these vertices are the endpoints of the paths). In the interest of reliability, it is desirable to have multiple disjoint paths between pairs of nodes in the network. Some ISPs provide more reliability than others, but they may charge their clients more.

The Network Reliability Optimization Problem (NROP) is now defined as follows. An instance is an undirected graph with n vertices $v_1, \ldots, v_n$, a non-negative integer weight on each edge, and an n-by-n symmetric matrix R_{ij}. The objective is to find a subset S of the edges such that the total cost of the edges in S is minimized, with the requirement that for every pair of vertices v_i and v_j, there are at least R_{ij} disjoint paths from v_i to v_j such that all paths use only edges in S.

You've been hired as a summer intern to develop an efficient algorithm for solving NROP. After working on an algorithm for awhile, your team conjectures that the problem is NP-hard. Here is your task:

1. Define the decision version of this problem, which we will call NRDP;
2. Prove that NRDP is NP-hard via a reduction from an NP-hard problem from lecture.

Solution: 1) To show a decision version of the problem: If there is an undirected graph representing the network called G such that for G: each edge has a non-negative integer weight and there is an n by n symmetric matrix, then does there exist a subset S of the edges for some cost capacity C such that the total cost of the edges in s is minimized (i.e. is at most C) and for every pair of vertices v_i and v_j, there are at least R_{ij} disjoint paths from v_i to v_j such that all paths use only edges in S.
2) Using the HamCycle we can show a reduction from the HamCycle to the NRDP problem. Showing the turing reduction, we can define it as f(G(V,E)) for some input cycle in HamCycle.
f(G(V,E)):
1) for each edge in G_1 have a weight of 1 and set G_1 of the NRDP problem to be the same as V and E for G.
2) set the cost capacity to $|V| - 1$ for the number of vertices in V.
3) set R_{ij} to 1 for all pairs so that there is at least a disjoint path between each i and j in R.
4) return NRDP (G,R,C)
This is polynomial time as we can see that assigning G to G_1 and setting R_{ij} only takes constant or linear time, which shows that the time complexity for the reduction can be

done in polynomial time.
To prove this from HamCycle to NRDP, we know that if G has a cycle, then all edges in that cycle can form a subset where the total cost of the subset is the vertices - 1 which is solved in the problem as we do not exceed that cost. Since it is a cycle, we know there is at least one disjoint path between every vertex pair and that indicates that the NRDP problem can be solved with the cycle G. For the other way from NRDP to HamCycle, if there is a subset where the cost of the edges is at most C and for all vertex pairs there is at least one disjoint path, then the graph must have a Hamiltonian cycle because it there wasn't a cycle, then at least one vertex pair would have no disjoint path which would not meet the NRDP requirement. Therefore since this reduction is correct we can prove NRDP is NP-Hard.